КОНТРОЛЬНЫЕ ВОПРОСЫ — Лабораторная работа №8

1. Что такое JSON и чем он отличается от объекта JavaScript?
JSON — это просто текст, записанный по определённым правилам, чтобы им можно было обмениваться между программами и хранить его. Объект JavaScript — это живая структура в памяти, в ней могут быть функции, методы, undefined. В JSON всё строже: ключи только в двойных кавычках, нет функций и комментариев, значения — строка, число, true/false, null, массив или объект. Если сказать коротко: объект — это то, с чем работает программа, а JSON — то, во что его превращают, чтобы сохранить или передать.

2. Для чего нужны JSON.stringify() и JSON.parse()?
stringify превращает объект или массив в строку, parse делает наоборот — из строки собирает объект обратно. Без них в хранилище нормально ничего не положить, потому что туда можно писать только строки.

3. Почему localStorage хранит строки?
Так устроен сам Web Storage: это хранилище пар «ключ — строка». Если записать туда объект напрямую, он превратится в "[object Object]" и данные потеряются. Поэтому перед записью объект переводят в строку через stringify, а при чтении возвращают через parse.

4. Чем localStorage отличается от sessionStorage?
Сроком жизни. localStorage остаётся даже после закрытия браузера и компьютера, пока его не очистят вручную или кодом. sessionStorage живёт только в одной вкладке: обновление страницы данные переживают, а после закрытия вкладки они пропадают. Ещё у каждой вкладки свой sessionStorage, а localStorage общий для всех вкладок одного сайта.

5. Как сохранить массив объектов?
Сначала сделать из него строку, потом записать по ключу:
const tasks = [{ id: 1, text: "Сделать лабу", done: false }];
localStorage.setItem("tasks", JSON.stringify(tasks));
Чтобы достать обратно: JSON.parse(localStorage.getItem("tasks")).

6. Что возвращает getItem() для отсутствующего ключа?
null. Это важно проверять: если сразу отдать null в JSON.parse, получится null, а не массив, и код может упасть на методах вроде forEach. Поэтому пишут так: JSON.parse(localStorage.getItem(key) || "[]") или проверяют результат отдельно.

7. Как удалить запись по ключу?
localStorage.removeItem("ключ"); — удалит одну запись, остальные не тронет.

8. Как очистить хранилище текущего источника?

localStorage.clear(); — стирает все записи этого сайта (источника: протокол + домен + порт). Данные других сайтов не затрагиваются. Для вкладки то же самое с sessionStorage.clear().

9. Почему обработка ошибок JSON.parse() важна?
Потому что в хранилище может оказаться что угодно: пользователь мог вручную поправить значение в DevTools, запись могла обрезаться, в этом ключе мог остаться формат от старой версии приложения. Если JSON битый, parse выбросит исключение и весь скрипт остановится, страница будет пустой. Поэтому parse оборачивают в try...catch и в случае ошибки используют пустое значение по умолчанию, а ещё проверяют, что получился именно массив или объект, который ожидаем.

10. Как проверить данные в DevTools?
Открыть DevTools (F12), перейти во вкладку Application, слева найти раздел Storage → Local Storage (или Session Storage) и выбрать адрес сайта. Справа будет таблица ключей и значений. Там же можно изменить значение, удалить запись или очистить всё. Ещё можно ввести в консоли localStorage.getItem("ключ").

11. Почему нельзя хранить пароли и токены в Web Storage?
Данные там лежат открытым текстом, и любой JavaScript на странице может их прочитать. Если на сайте есть XSS-уязвимость или подключён чужой скрипт, он просто заберёт всё из localStorage. Кроме того, человек за этим же компьютером может открыть DevTools и увидеть значения. Для секретов лучше подходят защищённые cookie с флагом HttpOnly и хранение на сервере.

12. Что произойдёт после обновления страницы и после закрытия вкладки?
После обновления страницы и localStorage, и sessionStorage сохраняются, интерфейс просто заново строится из них. После закрытия вкладки sessionStorage стирается, а localStorage остаётся и будет на месте даже при следующем запуске браузера.